***BRACT's***

**Vishwakarma Institute of Technology, Pune**

**Practical Implementation Sheet**

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: C Batch: 2 Practical No: 1

Roll no: 44 PRN No: 12411367 Name of Student: **Sarthak Kokadwar**

**Subject Name : Deep Learning**

**Problem Statement** : Install and configure TensorFlow/Keras in Google Colab. Perform data preprocessing, normalization, train-test splitting, and data visualization on a sample dataset

## Algorithm :

- Open **Google Colab** and create a new notebook.
- Import the required libraries such as **TensorFlow, Keras, NumPy, Matplotlib**, and **Scikit-learn**.
- Verify that **TensorFlow** and **Keras** are installed and configured in the Colab environment.
- Load the **MNIST dataset** into the notebook.
- Separate the dataset into **training images, training labels, testing images, and testing labels**.
- Examine the dataset by checking its **shape, dimensions, and labels**.
- Perform **data preprocessing** by converting the image data into an appropriate numerical format.
- Normalize the pixel values by dividing each pixel value by **255**, scaling the range from **0–255** to **0–1**.
- If required, split the dataset into **training** and **testing** sets using an appropriate train-test split ratio.
- Visualize sample images from the dataset using **Matplotlib** to verify that the data has been loaded correctly.
- Display the corresponding labels for the visualized images.

- Confirm that the dataset is successfully preprocessed and ready for training a deep learning model.

## Code :

```
import tensorflow as tf
import matplotlib.pyplot as plt
```

```
(train_images, train_labels), (test_images, test_labels) =
tf.keras.datasets.fashion_mnist.load_data()
train_images = train_images /255.0
test_images = test_images / 255.0
print(train_images.shape)
print(test_images.shape)
print(train_labels)
plt.imshow(train_images[0], cmap='gray')
plt.show()
my_model = tf.keras.models.Sequential()
my_model.add(tf.keras.layers.Flatten(input_shape=(28, 28)))
my_model.add(tf.keras.layers.Dense(128, activation='relu'))
my_model.add(tf.keras.layers.Dense(10, activation='softmax'))

my_model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
metrics=['accuracy'])

my_model.fit(train_images, train_labels, epochs=3)
val_loss, val_acc = my_model.evaluate(test_images, test_labels)
print('Test accuracy: ', val_acc)
my_model.save("my_mnist_model.keras")
```

## Output :

1. **Dataset Loaded Successfully :**

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz4422102/4422102 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

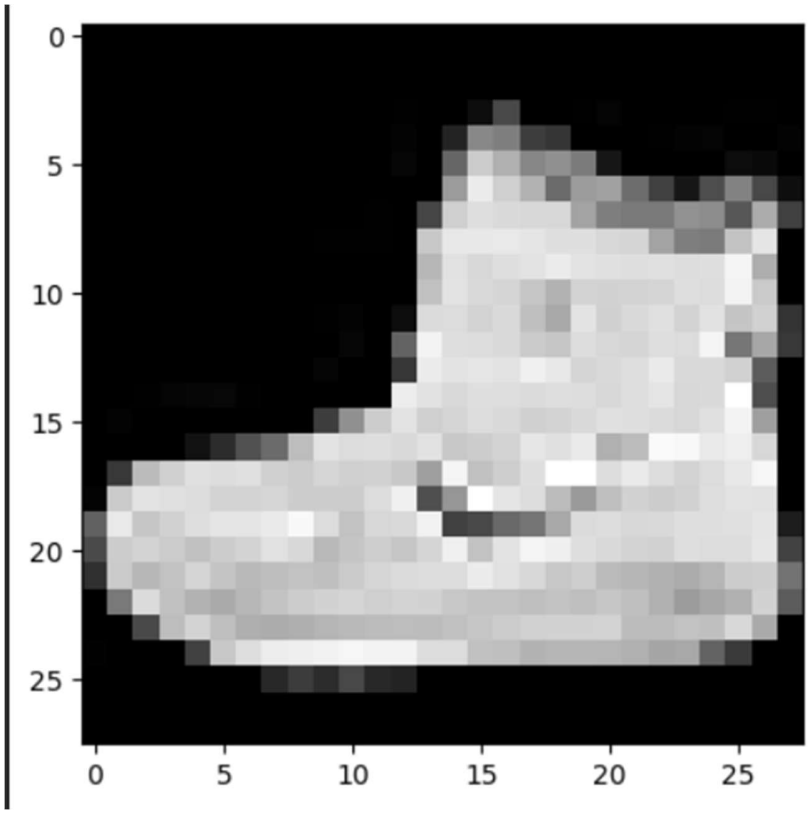
2. **Maximum Pixel Value Before Normalization : 255.0**

**3. Dataset Shape :**

Training Images Shape : (60000, 28, 28)

Testing Images Shape : (10000, 28, 28)

**4. Sample Image :**




## 5. Model Training :

```
Epoch 1/3
1875/1875 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.8261 -
loss: 0.4957
Epoch 2/3
1875/1875 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.8648 -
loss: 0.3751
Epoch 3/3
1875/1875 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8768 -
loss: 0.3374
<keras.src.callbacks.history.History at 0x7d34250a25d0>
```

## 6. Test Accuracy :

```
313/313 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8668
- loss: 0.3794
Test accuracy:  0.8668000102043152
```